

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования

**«Пермский национальный исследовательский политехнический
университет»**

Электротехнический факультет

Выпускающая кафедра: Информационные технологии и
автоматизированные системы (ИТАС)

Направление подготовки: 09.03.04 Программная инженерия

ОТЧЕТ

Лабораторная работа №13

«Стандартные обобщенные алгоритмы библиотеки STL.»

По дисциплине «Основы алгоритмизации и программирования»

Вариант 15

Выполнил: студент группы РИС-25-2Б

Ильинов Фёдор Михайлович

Принял: Доц. Полякова О. А.

Пермь 2026

ПОСТАНОВКА ЗАДАЧИ

Задача 1.

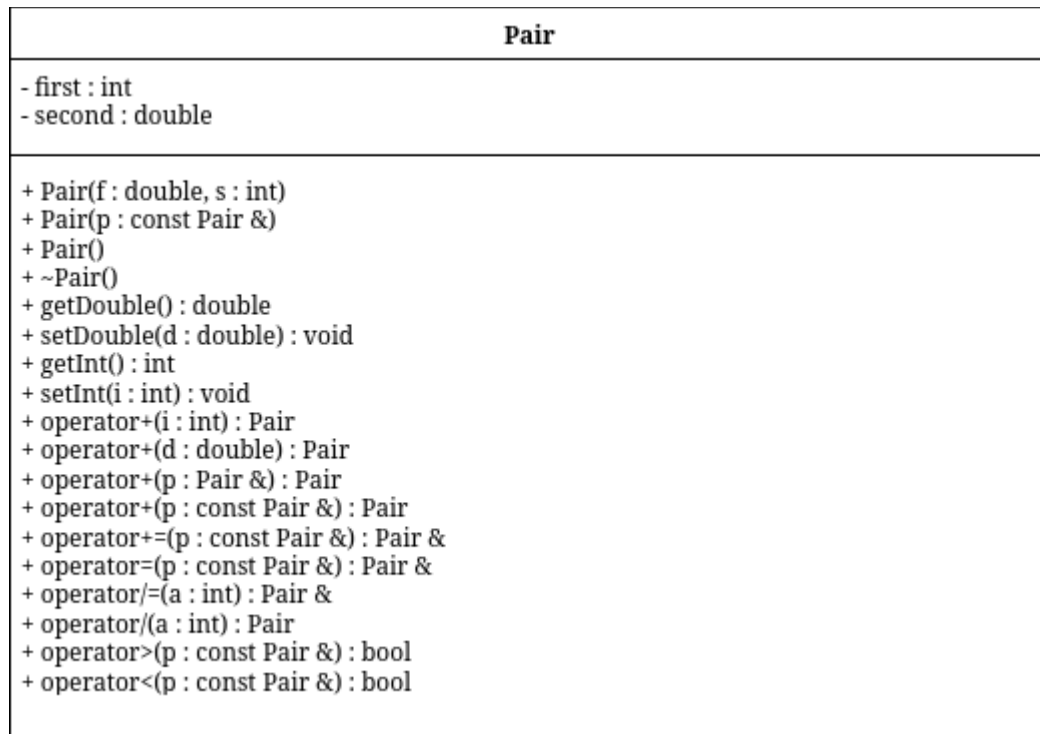
1. Создать последовательный контейнер.
2. Заполнить его элементами пользовательского типа (тип указан в варианте). Для пользовательского типа перегрузить необходимые операции.
3. Заменить элементы в соответствии с заданием (использовать алгоритмы `replace_if()`, `replace_copy()`, `replace_copy_if()`, `fill()`).
4. Удалить элементы в соответствии с заданием (использовать алгоритмы `remove()`, `remove_if()`, `remove_copy_if()`, `remove_copy()`)
5. Отсортировать контейнер по убыванию и по возрастанию ключевого поля (использовать алгоритм `sort()`).
6. Найти в контейнере заданный элемент (использовать алгоритмы `find()`, `find_if()`, `count()`, `count_if()`).
7. Выполнить задание варианта для полученного контейнера (использовать алгоритм `for_each()`).
8. Для выполнения всех заданий использовать стандартные алгоритмы библиотеки STL. Задача 2.
9. Создать адаптер контейнера.
10. Заполнить его элементами пользовательского типа (тип указан в варианте). Для пользовательского типа перегрузить необходимые операции.
11. Заменить элементы в соответствии с заданием (использовать алгоритмы `replace_if()`, `replace_copy()`, `replace_copy_if()`, `fill()`).
12. Удалить элементы в соответствии с заданием (использовать алгоритмы `remove()`, `remove_if()`, `remove_copy_if()`, `remove_copy()`)

13. Отсортировать контейнер по убыванию и по возрастанию ключевого поля (использовать алгоритм `sort()`).
14. Найти в контейнере элемент с заданным ключевым полем (использовать алгоритмы `find()`, `find_if()`, `count()`, `count_if()`).
15. Выполнить задание варианта для полученного контейнера (использовать алгоритм `for_each()`).
16. Для выполнения всех заданий использовать стандартные алгоритмы библиотеки STL. Задача 3
17. Создать ассоциативный контейнер.
18. Заполнить его элементами пользовательского типа (тип указан в варианте). Для пользовательского типа перегрузить необходимые операции.
19. Заменить элементы в соответствии с заданием (использовать алгоритмы `replace_if()`, `replace_copy()`, `replace_copy_if()`, `fill()`).
20. Удалить элементы в соответствии с заданием (использовать алгоритмы `remove()`, `remove_if()`, `remove_copy_if()`, `remove_copy()`).
21. Отсортировать контейнер по убыванию и по возрастанию ключевого поля (использовать алгоритм `sort()`).
22. Найти в контейнере элемент с заданным ключевым полем (использовать алгоритмы `find()`, `find_if()`, `count()`, `count_if()`).
23. Выполнить задание варианта для полученного контейнера (использовать алгоритм `for_each()`).
24. Для выполнения всех заданий использовать стандартные алгоритмы библиотеки STL.

АНАЛИЗ.

1. Воспользуемся функциями из работы №11, изменяя логику там, где это возможно, на алгоритмы STL.
2. Добавим вывод отсортированного контейнера по возрастанию (asc) и по убыванию (desc) для демонстрации работы функции sort().

UML-ДИАГРАММА



Код Pair.h

```
#ifndef ILINOV_FEDOR_RIS_25_2B_LABS_PSTU_CS_FRACTION_H
#define ILINOV_FEDOR_RIS_25_2B_LABS_PSTU_CS_FRACTION_H
#include <ostream>

class Pair {
    int first;
    double second;

public:
    Pair(double, int);
    Pair(const Pair &);
    Pair();
    ~Pair();
    double getDouble() const;
    void setDouble(double);
    int getInt() const;
    void setInt(int);
    Pair &operator=(Pair const &p);
    Pair operator+(Pair &p);
    Pair operator+(int);
    Pair operator+(double);
    Pair& operator+=(Pair const &p);
    Pair& operator/=(int a);
    Pair operator/(int a) const;
    bool operator>(Pair const &p) const;
    bool operator<(Pair const &p) const;
    bool operator==(const Pair &p) const;
    friend std::ostream &operator<<(std::ostream &, const Pair &);
    friend std::istream &operator>>(std::istream &, Pair &);
};

#endif //ILINOV_FEDOR_RIS_25_2B_LABS_PSTU_CS_FRACTION_H
```

КОД Pair.cpp

```
#include "Pair.h"
#include <iostream>

Pair::Pair(double a, int b) {
    first = b;
    second = a;
}

Pair::Pair(const Pair &f) {
    second = f.second;
    first = f.first;
}

Pair::Pair() {
    second = 0;
    first = 0;
}

Pair::~Pair() {
    second = 0;
    first = 0;
}

double Pair::getDouble() const {
    return second;
}

void Pair::setDouble(double a) {
    second = a;
}

int Pair::getInt() const {
    return first;
}

void Pair::setInt(int a) {
    first = a;
}

Pair &Pair::operator=(Pair const &p) {
    second = p.second;
    first = p.first;
    return *this;
}
```

```
}
```

```
bool Pair::operator<(Pair const &p) const {  
    if (this->getInt() < p.getInt())  
        return true;  
    if (this->getInt() == p.getInt())  
        return this->getDouble() < p.getDouble();  
    return false;  
}
```

```
bool Pair::operator>(Pair const &p) const {  
    return p < *this;  
}
```

```
Pair & Pair::operator+=(Pair const &p) {  
    this->first += p.first;  
    this->second += p.second;  
    return *this;  
}
```

```
Pair & Pair::operator/=(int a) {  
    this->first /= a;  
    this->second /= a;  
    return *this;  
}
```

```
Pair Pair::operator/(int a) const {  
    return Pair(  
        this->second / a,  
        this->first / a  
    );  
}
```

```
bool Pair::operator==(const Pair &p) const {  
    return (this->first == p.first && this->second == p.second);  
}
```

```
std::ostream &operator<<(std::ostream &stream, const Pair &p) {  
    std::cout << p.first << ":" << p.second;  
    return stream;  
}
```

```
std::istream &operator>>(std::istream &stream, Pair &p) {  
    std::cout << "Double? ";  
    stream >> p.second;  
}
```

```
std::cout << "Int? ";  
stream >> p.first;  
return stream;  
}
```

```
Pair Pair::operator+(Pair &p) {  
    Pair second(*this);  
    second.second += p.second;  
    second.first += p.first;  
    return second;  
}
```

```
Pair Pair::operator+(int a) {  
    Pair tmp(*this);  
    tmp.first += a;  
    return tmp;  
}
```

```
Pair Pair::operator+(double a) {  
    Pair tmp(*this);  
    tmp.second += a;  
    return tmp;  
}
```


КОД main.cpp

```
#include <iostream>
#include <list>
#include <map>
#include <queue>
#include <random>
#include <algorithm>
#include <numeric>
#include "Pair.h"

std::random_device rd;
std::mt19937 gen(rd());

template<class T>
void printItem(const T& i) { std::cout << i << ' '; }

void printPair(const std::pair<const Pair, int>& kv) { std::cout << kv.first << ' '; }

Pair extractKey(const std::pair<const Pair, int>& kv) { return kv.first; }

template<class T>
void printContainerAfterSubtask(const T& l) {
    std::cout << "\t\t";
    std::for_each(l.begin(), l.end(), printItem<typename T::value_type>);
    std::cout << '\n';
}

template<class T>
T subtask1(const T& t) {
    T l = t;
    std::cout << "\tsubtask 1\n";
    using Item = typename T::value_type;

    Item avg = Item();
    std::for_each(l.begin(), l.end(), [&avg](const Item& i){ avg += i; });
    avg /= (int)l.size();
    l.push_back(avg);

    std::cout << "\t\t" << "avg: " << avg << '\n';

    return l;
}

template<class T>
```

```

T subtask2(const T& t) {
    T l = t;

    std::cout << "\tsubtask 2\n";
    using Item = typename T::value_type;

    std::uniform_int_distribution<> dist(0, 2);

    std::vector<Item> forDelete;
    std::remove_copy_if(l.begin(), l.end(), std::back_inserter(forDelete),
        [&](const Item&){ return dist(gen) != 0; });

    std::cout << "\t\tFor delete:\n\t\t";

    std::for_each(forDelete.begin(), forDelete.end(), printItem<Item>);

    if constexpr (std::is_same_v<T, std::list<Item>>) {
        std::for_each(forDelete.begin(), forDelete.end(), [&](const Item& i){ l.remove(i); });
    } else {
        std::for_each(forDelete.begin(), forDelete.end(), [&](const Item& i){
            l.erase(std::remove(l.begin(), l.end(), i), l.end());
        });
    }
    std::cout << '\n';

    return l;
}

template<class T>
T subtask3(const T& t) {
    T l = t;

    std::cout << "\tsubtask 3\n";
    using Item = typename T::value_type;

    auto mn = *std::min_element(l.begin(), l.end());
    auto mx = *std::max_element(l.begin(), l.end());
    Item avg = (mn + mx) / 2;

    std::for_each(l.begin(), l.end(), [&avg](Item& i){ i += avg; });

    std::cout << "\t\t(min + max) / 2 = " << avg << '\n';

    return l;
}

```

```

void task_1() {
    std::cout << "task 1\n";

    std::list<double> l;
    std::uniform_int_distribution<> dist(-99, 99);
    for (int i = 0; i < 10; i++)
        l.push_back((double) dist(gen) / 10);

    std::cout << '\t';
    std::for_each(l.begin(), l.end(), printItem<double>);
    std::cout << '\n';

    l.sort();
    std::cout << "\t[asc]: "; std::for_each(l.begin(), l.end(), printItem<double>); std::cout << '\n';
    l.sort(std::greater<double>());
    std::cout << "\t[desc]: "; std::for_each(l.begin(), l.end(), printItem<double>); std::cout << '\n';

    l = subtask1(l);
    printContainerAfterSubtask(l);

    l = subtask2(l);
    printContainerAfterSubtask(l);

    l = subtask3(l);
    printContainerAfterSubtask(l);
}

```

```

template<class T>
std::priority_queue<T> copyBiListToSTLPriorityQueue(const std::vector<T>& bl) {
    std::priority_queue<T> q;
    std::for_each(bl.begin(), bl.end(), [&q](const T& i){ q.push(i); });
    return q;
}

```

```

template<class T>
std::vector<T> copySTLPriorityQueueToBiList(std::priority_queue<T>& q) {
    std::vector<T> bl;
    while (!q.empty()) {
        bl.push_back(q.top());
        q.pop();
    }
    q = copyBiListToSTLPriorityQueue(bl);
    return bl;
}

```

```

template<class T>
void printSTLPriorityQueue(std::priority_queue<T>& q) {
    auto v = copySTLPriorityQueueToBiList(q);
    std::for_each(v.begin(), v.end(), printItem<T>);
    std::cout << '\n';
}

void task_2() {
    std::cout << "task 4\n";
    std::priority_queue<Pair> q;

    std::uniform_int_distribution<> dist(0, 9);
    for (int i = 0; i < 10; i++) {
        Pair p((double)dist(gen) / 10, dist(gen));
        q.push(p);
    }

    std::cout << '\t';
    printSTLPriorityQueue(q);

    {
        auto sv = copySTLPriorityQueueToBiList(q);
        std::sort(sv.begin(), sv.end());
        std::cout << "\t[asc]: "; std::for_each(sv.begin(), sv.end(), printItem<Pair>); std::cout << '\n';
        std::sort(sv.begin(), sv.end(), std::greater<Pair>());
        std::cout << "\t[desc]: "; std::for_each(sv.begin(), sv.end(), printItem<Pair>); std::cout << '\n';
    }

    auto v = copySTLPriorityQueueToBiList(q);
    v = subtask1(v);
    q = copyBiListToSTLPriorityQueue(v);
    std::cout << "\t\t";
    printSTLPriorityQueue(q);

    v = copySTLPriorityQueueToBiList(q);
    v = subtask2(v);
    q = copyBiListToSTLPriorityQueue(v);
    std::cout << "\t\t";
    printSTLPriorityQueue(q);

    v = copySTLPriorityQueueToBiList(q);
    v = subtask3(v);
    q = copyBiListToSTLPriorityQueue(v);
    std::cout << "\t\t";
}

```

```

printSTLPriorityQueue(q);

std::cout << '\n';
}

void task_30 {
    std::cout << "task 4\n";
    std::map<Pair, int> mp;

    std::uniform_int_distribution<> dist(0, 9);
    for (int i = 0; i < 10; i++) {
        Pair p((double)dist(gen) / 10, dist(gen));
        mp[p] = dist(gen);
    }

    std::cout << '\t';
    std::for_each(mp.begin(), mp.end(), printPair);
    std::cout << '\n';

    std::vector<Pair> v;
    std::transform(mp.begin(), mp.end(), std::back_inserter(v), extractKey);

    std::sort(v.begin(), v.end());
    std::cout << "t[asc]: "; std::for_each(v.begin(), v.end(), printItem<Pair>); std::cout << '\n';
    std::sort(v.begin(), v.end(), std::greater<Pair>());
    std::cout << "t[desc]: "; std::for_each(v.begin(), v.end(), printItem<Pair>); std::cout << '\n';

    v = subtask1(v);
    std::cout << "t\t";
    std::for_each(v.begin(), v.end(), printItem<Pair>);
    std::cout << '\n';

    v = subtask2(v);
    std::cout << "t\t";
    std::for_each(v.begin(), v.end(), printItem<Pair>);
    std::cout << '\n';

    v = subtask3(v);
    std::cout << "t\t";
    std::for_each(v.begin(), v.end(), printItem<Pair>);
    std::cout << '\n';

    std::cout << '\n';
}

```

```
int main() {
    task_1();
    task_2();
    task_3();
}
```

РЕЗУЛЬТАТ ВЫПОЛНЕНИЯ

task 1

-0.6 -4.9 6 -2.7 -0.3 3.9 -6.5 5.7 -4 -8.2

[asc]: -8.2 -6.5 -4.9 -4 -2.7 -0.6 -0.3 3.9 5.7 6

[desc]: 6 5.7 3.9 -0.3 -0.6 -2.7 -4 -4.9 -6.5 -8.2

subtask 1

avg: -1.16

6 5.7 3.9 -0.3 -0.6 -2.7 -4 -4.9 -6.5 -8.2 -1.16

subtask 2

For delete:

-4.9 -1.16

6 5.7 3.9 -0.3 -0.6 -2.7 -4 -6.5 -8.2

subtask 3

$(\min + \max) / 2 = -1.1$

4.9 4.6 2.8 -1.4 -1.7 -3.8 -5.1 -7.6 -9.3

task 4

9:0.9 9:0.2 8:0.5 8:0.3 6:0.8 6:0.7 5:0.1 1:0.8 1:0.4 1:0

[asc]: 1:0 1:0.4 1:0.8 5:0.1 6:0.7 6:0.8 8:0.3 8:0.5 9:0.2 9:0.9

[desc]: 9:0.9 9:0.2 8:0.5 8:0.3 6:0.8 6:0.7 5:0.1 1:0.8 1:0.4 1:0

subtask 1

avg: 5:0.47

9:0.9 9:0.2 8:0.5 8:0.3 6:0.8 6:0.7 5:0.47 5:0.1 1:0.8 1:0.4 1:0

subtask 2

For delete:

5:0.47 1:0.4 1:0

9:0.9 9:0.2 8:0.5 8:0.3 6:0.8 6:0.7 5:0.1 1:0.8

subtask 3

$$(\min + \max) / 2 = 5:0.85$$

14:1.75 14:1.05 13:1.35 13:1.15 11:1.65 11:1.55 10:0.95 6:1.65

task 4

0:0 0:0.2 2:0.2 4:0.2 4:0.8 4:0.9 5:0.1 6:0.2 7:0 9:0.3

[asc]: 0:0 0:0.2 2:0.2 4:0.2 4:0.8 4:0.9 5:0.1 6:0.2 7:0 9:0.3

[desc]: 9:0.3 7:0 6:0.2 5:0.1 4:0.9 4:0.8 4:0.2 2:0.2 0:0.2 0:0

subtask 1

avg: 4:0.29

9:0.3 7:0 6:0.2 5:0.1 4:0.9 4:0.8 4:0.2 2:0.2 0:0.2 0:0 4:0.29

subtask 2

For delete:

5:0.1 0:0.2 4:0.29

9:0.3 7:0 6:0.2 4:0.9 4:0.8 4:0.2 2:0.2 0:0

subtask 3

$$(\min + \max) / 2 = 4:0.15$$

13:0.45 11:0.15 10:0.35 8:1.05 8:0.95 8:0.35 6:0.35 4:0.15